COMSM0122 Workshop2 reflection log

Part 1: The Trust Problem

```
E:\UoB_all\System_S3c\week5\BFT\bft-lab>node.exe scenarios/honest.js

====================
Proposal: 100
====================
A voted 100
B voted 100
C voted 100
D voted 100

Vote Tally
┌─────────┬────────┐
│ (index) │ Values │
├─────────┼────────┤
│   100   │   4    │
└─────────┴────────┘
Required Quorum = 3

COMMITTED: 100
```

In the honest scenario, Nodes A, B, C, and D all voted for "100," exceeding the system's 3-vote quorum; therefore, 100 was ultimately committed, and consensus was reached. Quorum specifies the minimum number of votes required for a commit; it ensures that a value must receive support from a sufficient number of nodes before it can be committed. In this scenario, each node contributes only one vote, so the outcome is determined by the collective votes of multiple nodes and the protocol rules.

Part 2: Byzantine Behaviour

```
E:\UoB_all\System_S3c\week5\BFT\bft-lab>node.exe scenarios/byzantine.js

====================
Proposal: 100
====================
A voted 100
B voted 100
C voted 100
D voted 50

Vote Tally
┌─────────┬────────┐
│ (index) │ Values │
├─────────┼────────┤
│   50    │   1    │
│   100   │   3    │
└─────────┴────────┘
Required Quorum = 3

COMMITTED: 100
```

In the byzantine scenario, Nodes A, B, and C all voted for 100, while Node D voted for 50, exhibiting Byzantine behaviour that deviated from the requirement. Ultimately, 100 received

three votes and 50 received only one vote; the system correctly committed 100 based on the quorum. The protocol simply counts the valid votes for each value and checks whether a quorum has been reached; based on the 3f+1 rule, the protocol assumes that, out of 4 nodes, at most 1 is a Byzantine node, while the remaining 3 are honest nodes.

Part 3: Equivocation

```
E:\UoB_all\System_S3c\week5\BFT\bft-lab>node.exe scenarios/equivocation.js
{
  nodeId: 'D',
  value: 100,
  signature: 'fc9daaeaf50cc2daf1b9a23cef673a4d7e0b56e25422ce147744738fa75e8017'
}
{
  nodeId: 'D',
  value: 50,
  signature: '95298f81a6212895a9b19fc91bdb63b34aea36c813cb5ff8033c0d9f8680872e'
}
{
  nodeId: 'D',
  value: 75,
  signature: '464c60f5969fc779697d7c23bbfc9493bb0b6f1705dd592d02a0de2e22abefbb'
}
```

In the equivocation scenario, node D generated three messages, each claiming a value of 100, 50, and 75, respectively, and each accompanied by a signature; different receivers will see different statements from D. D uses its own valid key to sign different messages. However, the screenshot only shows the signature field and does not display the verification result; it can only prove that the message originated from the corresponding key holder and has not been modified. In this scenario, the node that receives 100 is unaware that D has also sent 50 and 75 to others, which increases the difficulty of coordination and detection; the solution is for nodes to exchange signed messages to detect conflicts.

Part 4: Cryptographic Evidence and Key Theft

```
E:\UoB_all\System_S3c\week5\BFT\bft-lab>node.exe scenarios/key_theft.js
Forged message
{
  nodeId: 'A',
  value: 999,
  signature: '669dd5970bf6d1d8ac7b2aaffbfd9d7159f19aeae7c3c45a6dd50bc1867161d0'
}
```

In the key theft scenario, after obtaining A's private key, the attacker uses it to sign a value of 999. Both this invalid message and a valid message sent by A himself contain the same structure of node_id, value, and a valid signature—so the verifier cannot determine whether the actual sender is A or the attacker. The protocol assumes that private keys are kept confidential, that only authorized nodes can use them, and that the binding between the key and the node's identity is trustworthy. If an attacker uses A's genuine private key to generate a signature that matches A's verification key—thereby passing the signature verification—the attacker can submit a valid but malicious vote impersonating A. In this case, A's identity should be considered compromised.

Part 5: Sybil Attacks and Membership Management

```
E:\UoB_all\System_S3c\week5\BFT\bft-lab>node.exe scenarios/sybil.js
Original validator set
```

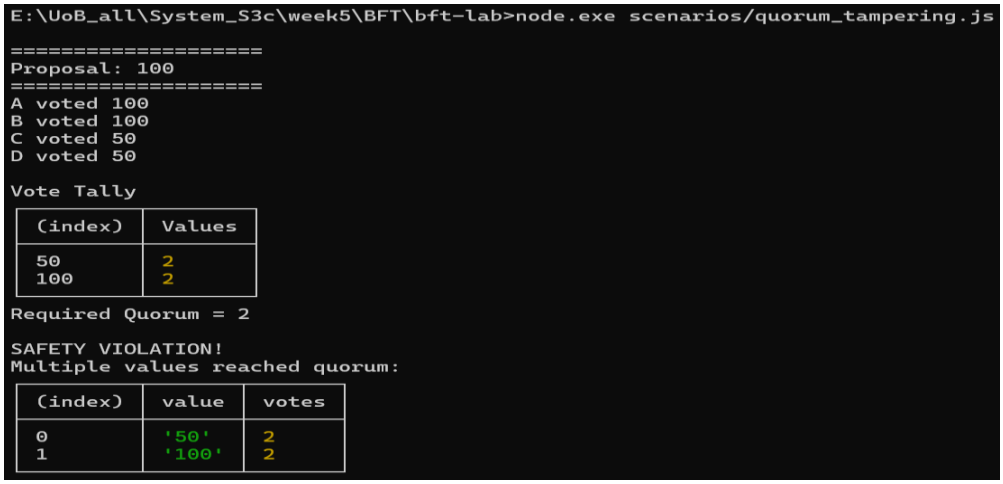
(index)	Values
0	'A'
1	'B'
2	'C'
3	'D'

After Sybil attack

(index)	Values
0	'A'
1	'B'
2	'C'
3	'D'
4	'X1'
5	'X2'
6	'X3'
7	'X4'

In the Sybil scenario, the attacker created four new identities: X1, X2, X3, X4 to gain additional influence; if each identity has one vote, the attacker now has 4 votes, or 50%. The attacker's 4 votes are not enough to submit a change to the result on their own, but by abstaining from voting, they can prevent the remaining 4 honest nodes from reaching the threshold. Typically, the system establishes certain rules and designates administrators to determine which nodes are eligible to vote. Consensus can only resolve the issue of how nodes form their opinions; it cannot determine who is eligible to vote. In a "one identity, one vote" system, a Sybil attacker could control the majority of voting power, potentially rendering the submission system untrustworthy.

Part 6: Quorum Reasoning and Safety

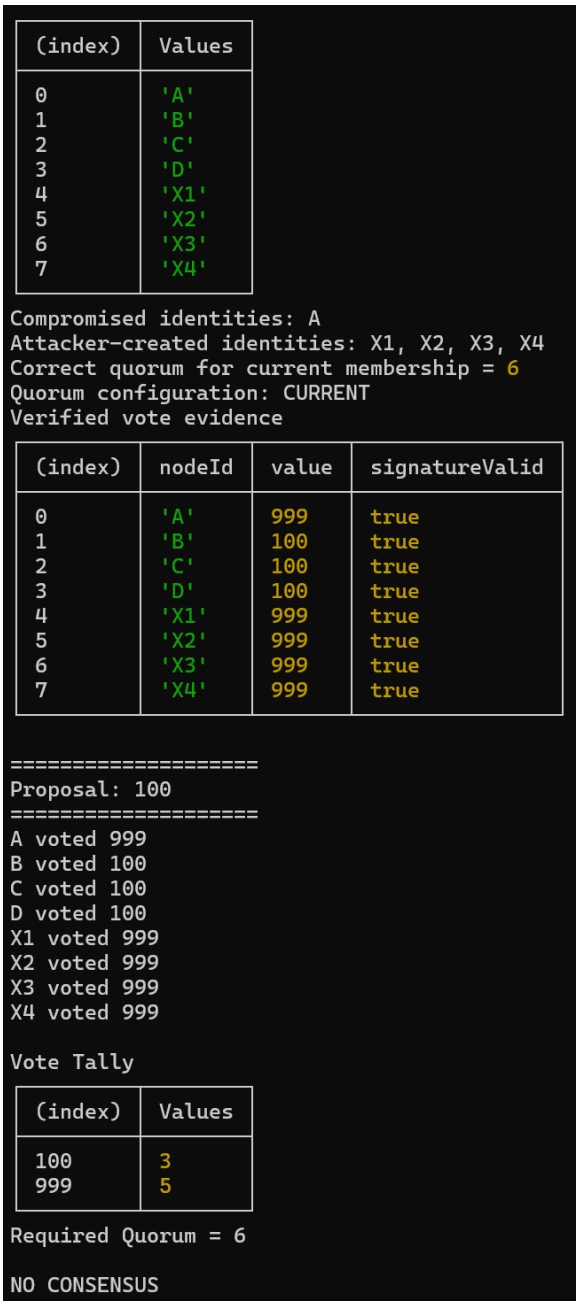


In the quorum_tampering scenario, both conflicting values meet the submission threshold, so the program outputs: “SAFETY VIOLATION! Multiple values reached quorum”, Because the 2-vote threshold allows four nodes to be divided into two completely non-overlapping voting groups, each of which can independently satisfy the submission conditions, the rules cannot prevent conflicting decisions. This situation violates consistency; for the same consensus instance, honest nodes should not submit different values. In this output, both 100 and 50 received 2 votes each, both meeting the revised threshold.

Part 7: Breaking BFT

This section will discuss scenarios in which multiple attack patterns coexist; assume that compromised nodes vote for 999, while trusted nodes vote for 100.

- Phase 1: Sybil injection + stolen A key + current quorum



(index)	Values
0	'A'
1	'B'
2	'C'
3	'D'
4	'X1'
5	'X2'
6	'X3'
7	'X4'

```
Compromised identities: A
Attacker-created identities: X1, X2, X3, X4
Correct quorum for current membership = 6
Quorum configuration: CURRENT
Verified vote evidence
```

(index)	nodeId	value	signatureValid
0	'A'	999	true
1	'B'	100	true
2	'C'	100	true
3	'D'	100	true
4	'X1'	999	true
5	'X2'	999	true
6	'X3'	999	true
7	'X4'	999	true

```
====================
Proposal: 100
====================
A voted 999
B voted 100
C voted 100
D voted 100
X1 voted 999
X2 voted 999
X3 voted 999
X4 voted 999

Vote Tally
```

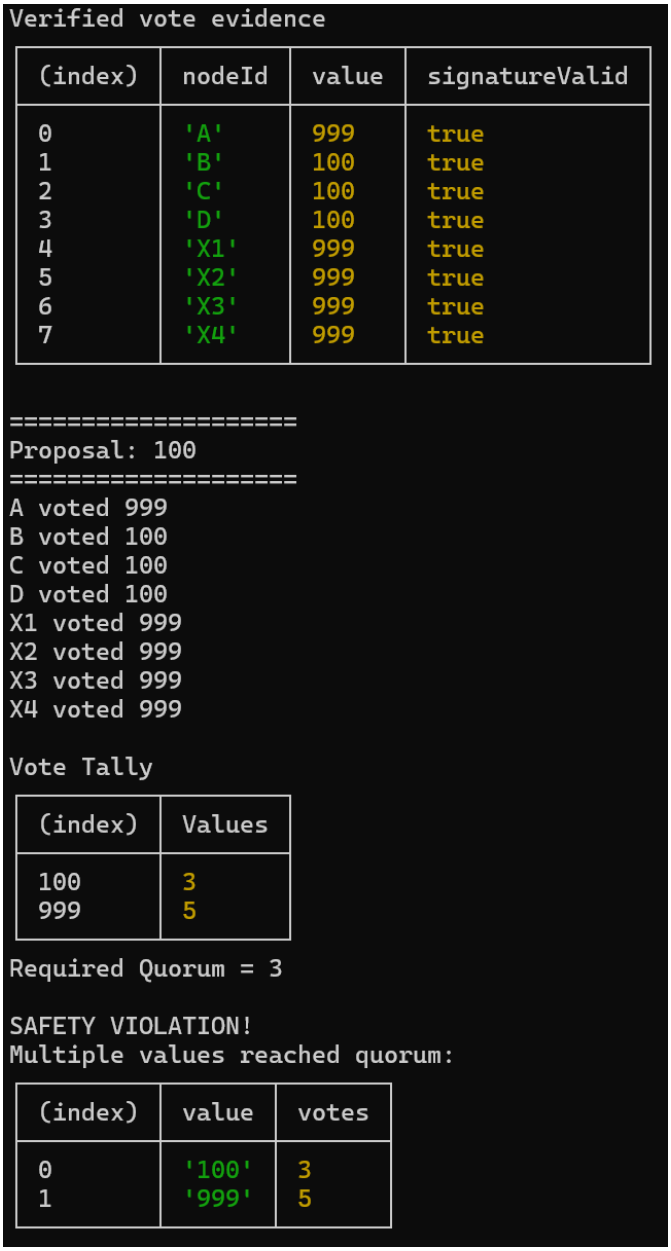
(index)	Values
100	3
999	5

```
Required Quorum = 6

NO CONSENSUS
```

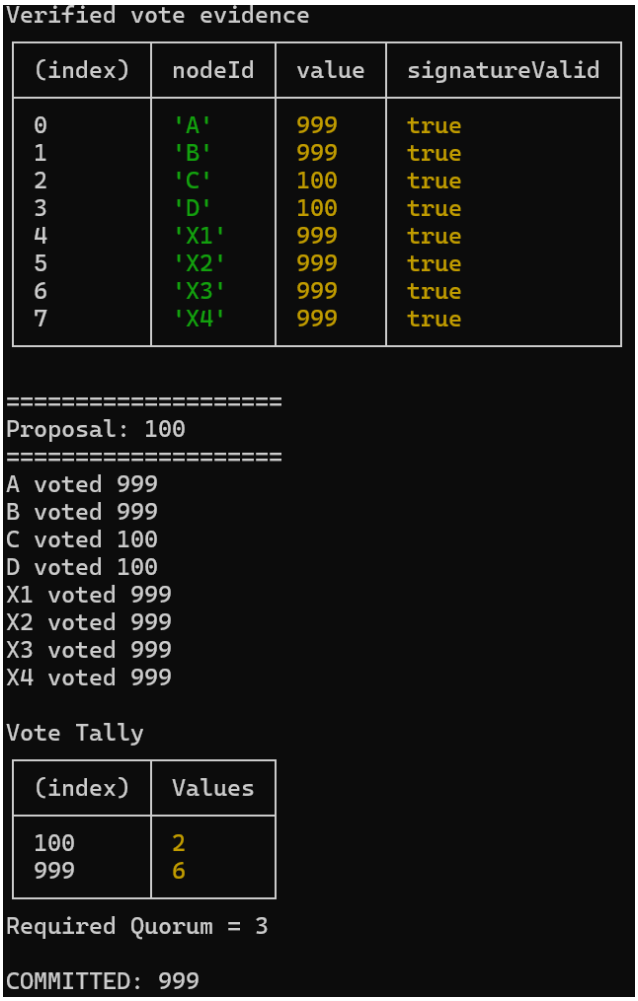
In this phase, the attacker adds four nodes—X1 through X4—and uses the stolen private key to impersonate A and cast votes; as the total number of nodes increases, the quorum is updated to 6 according to the program settings. The results show that B, C, and D vote for 100, while the impersonated A and X1 through X4 vote for 999. Since neither value reaches the quorum, the output is "NO CONSENSUS." These results demonstrate that uncontrolled member admission and key leaks can allow attackers to gain excessive voting power and block submissions in the current round. It should be noted that at this point, the system has exceeded the BFT fault-tolerance assumption ($3 \times 5 + 1 = 16$).

- Phase 2: Sybil injection + stolen A key + stale quorum



Under the member and voting conditions of Phase 1, intentionally retaining the quorum of 3 from the original four-node system would result in both 100 (3 votes) and 999 (5 votes) meeting the threshold; the program would then output a “SAFETY VIOLATION” as described in Part 6.

- Phase 3: Sybil injection + stolen A key + stolen B key + stale quorum

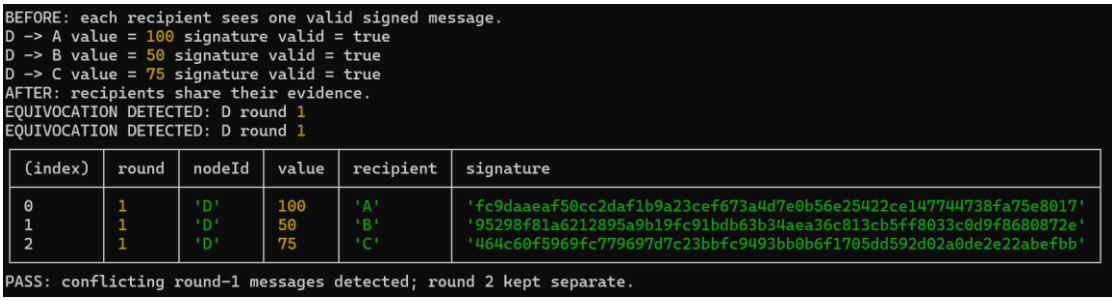


Building on Phase 2, the attacker further obtains B's key and impersonates B to vote for 999. At this point, C and D support 100, while the impersonated A, B, and X1–X4 support 999. Since the quorum remains at 3, only 999 meets the threshold, and the system ultimately outputs COMMITTED: 999.

Part 8: Hardening a BFT System

This part will attempt to apply some mitigation measures to different attack methods and examine the results.

- Detect Equivocation



D sends 100, 50, and 75 to A, B, and C, respectively, and the signatures on all three messages are valid. When each recipient views only their own message, they cannot

detect the conflict; after sharing the evidence, the system detects that D sent different values to different recipients in round n and outputs “EQUIVOCATION DETECTED: D round n.” The system successfully detects the issue and preserves the evidence, but it does not automatically prevent D from sending the messages.

- Strengthen Key Protection

```
BEFORE revocation: forged signature valid = true
REVOKED: A
REJECTED revoked identity: A
REJECTED revoked identity: A
REJECTED revoked identity: A

====================
Proposal: 100
====================
B voted 100
C voted 100
D voted 100

Vote Tally
```

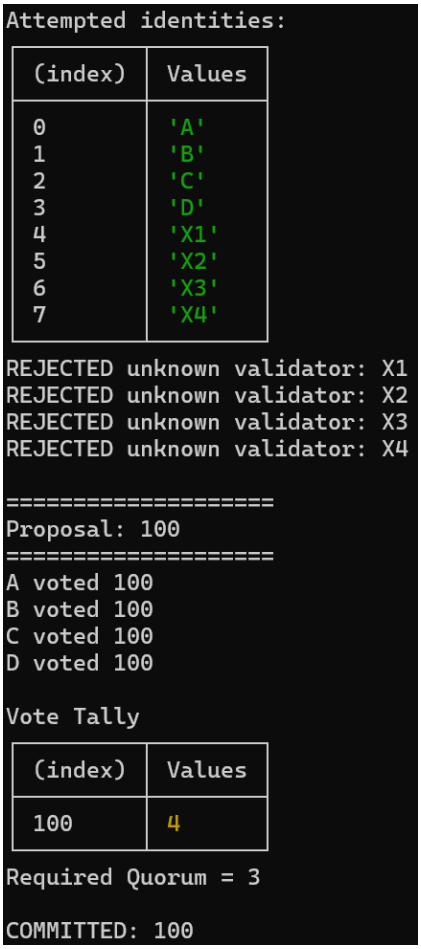
(index)	Values
100	3

```
Required Quorum = 3

COMMITTED: 100
COMMIT CERTIFICATE
Value = 100
Signed by: B C D
{
  value: 100,
  signatures: [
    {
      nodeId: 'B',
      value: 100,
      signature: '4f39484ebe84f05ce2de24aee57daa09bfa38d7e5183b6fb0f91c0b15557c58f'
    },
    {
      nodeId: 'C',
      value: 100,
      signature: '5bdf9f09b0cafb83b1f23f4ba11b013bcfa1ca2170aa93a119f15beec99c7909'
    },
    {
      nodeId: 'D',
      value: 100,
      signature: 'fc9daaeaf50cc2daf1b9a23cef673a4d7e0b56e25422ce147744738fa75e8017'
    }
  ]
}
PASS: revoked A cannot vote; B,C,D still commit with quorum 3.
```

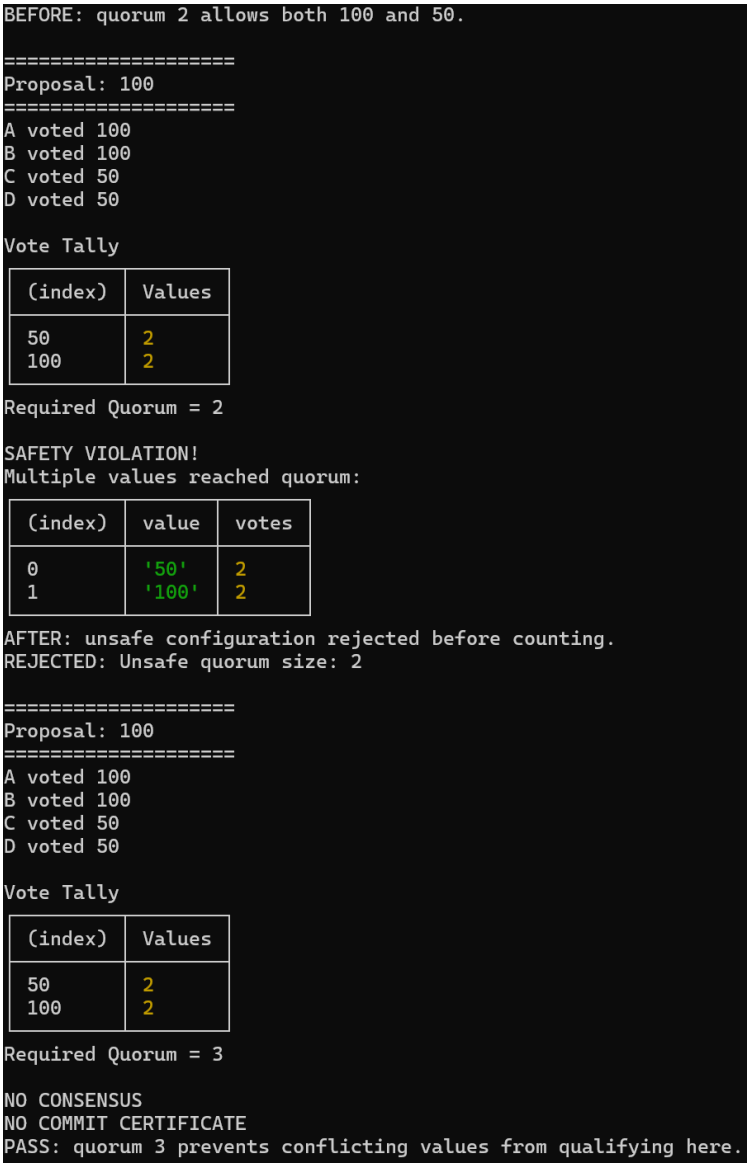
This operation attempts to mitigate the impact of key theft by revoking messages from the compromised node. Before revocation, an attacker impersonates A and uses the stolen key to generate valid messages; After executing `revoke(“A”)`, the system outputs “REJECTED revoked identity: A.” Even if the signature is valid, votes from that node are no longer counted; instead, the original quorum of 3 is reached with three votes from B, C, and D, ultimately committing 100. It is important to note that revocation also rejects votes sent by the genuine A using the old identity, and relies on external detection of the key theft to execute the revocation rather than self-detection.

- Prevent Sybil Attacks



To mitigate the Sybil attack, attempt to use “control member access. At this point, the attacker attempts to add X1–X4, but the predefined approval list still consists only of A–D; therefore, all four new identities are rejected, and only the four votes from A–D are counted, resulting in a final submission of 100; however, this mechanism does not prevent the attacker from creating new identities.

- Protect Quorum Rules



"Rejecting Unsafe Thresholds" is a method for Protect Quorum Rules. As shown in Part 6, before the defense mechanism is activated, 100 and 50 each have two votes, so the quorum is 2; both meet the threshold, resulting in a quorum conflict. After the check is enabled, the program strictly calculates the minimum threshold based on the settings and updates the quorum to 3. At this point, neither value can be committed, and consensus cannot be reached.

- Build a Commit Certificate

```
====================
Proposal: 100
====================
A voted 100
B voted 100
C voted 100
D voted 100

Vote Tally
```

(index)	Values
100	4

```
Required Quorum = 3

COMMITTED: 100
COMMIT CERTIFICATE
Value = 100
Signed by: A B C D
{
  value: 100,
  signatures: [
    {
      nodeId: 'A',
      value: 100,
      signature: '21a965bdad9ae4a678ebdcaba18993558e278a4dfc197de476ad6877a73769ad'
    },
    {
      nodeId: 'B',
      value: 100,
      signature: '4f39484ebe84f05ce2de24aee57daa09bfa38d7e5183b6fb0f91c0b15557c58f'
    },
    {
      nodeId: 'C',
      value: 100,
      signature: '5bdf9f09b0cafb83b1f23f4ba11b013bcfa1ca2170aa93a119f15beec99c7909'
    },
    {
      nodeId: 'D',
      value: 100,
      signature: 'fc9daaeaf50cc2daf1b9a23cef673a4d7e0b56e25422ce147744738fa75e8017'
    }
  ]
}
REJECTED invalid signature: A
PASS: valid certificate accepted.
PASS: insufficient, duplicate, altered-value and invalid-signature certificates rejected.
PASS: duplicate votes cannot inflate consensus counts.
```

To implement “Build a Commit Certificate,” you can try “preserving evidence for the commit,” meaning that once the number of votes from nodes reaches the threshold, the system not only outputs the commit value but also saves the identities and signatures of the nodes supporting that value. The “REJECTED invalid signature: A” in the results simulates a scenario where a key has been stolen, rather than a normal commit failure caused by a programming issue.